

# Ejercicio 7

## Enunciado

Escribir en la pantalla la palabra microprocesador y a continuación la misma palabra invertida de derecha a izquierda.

## Algoritmo

El algoritmo consiste en tomar a la palabra como una cadena de caracteres almacenada en una variable. Declarar otra variable para la cadena de caracteres resultante. Recorrer la primera cadena y armar la segunda ubicando cada carácter de derecha a izquierda. Finalmente imprimir las dos cadenas.

## Pseudocódigo

DIRECTIVAS:

.model small

.stack 64

.DATA

cadena 1 de origen y

cadena 2 invertida (reservar espacio)

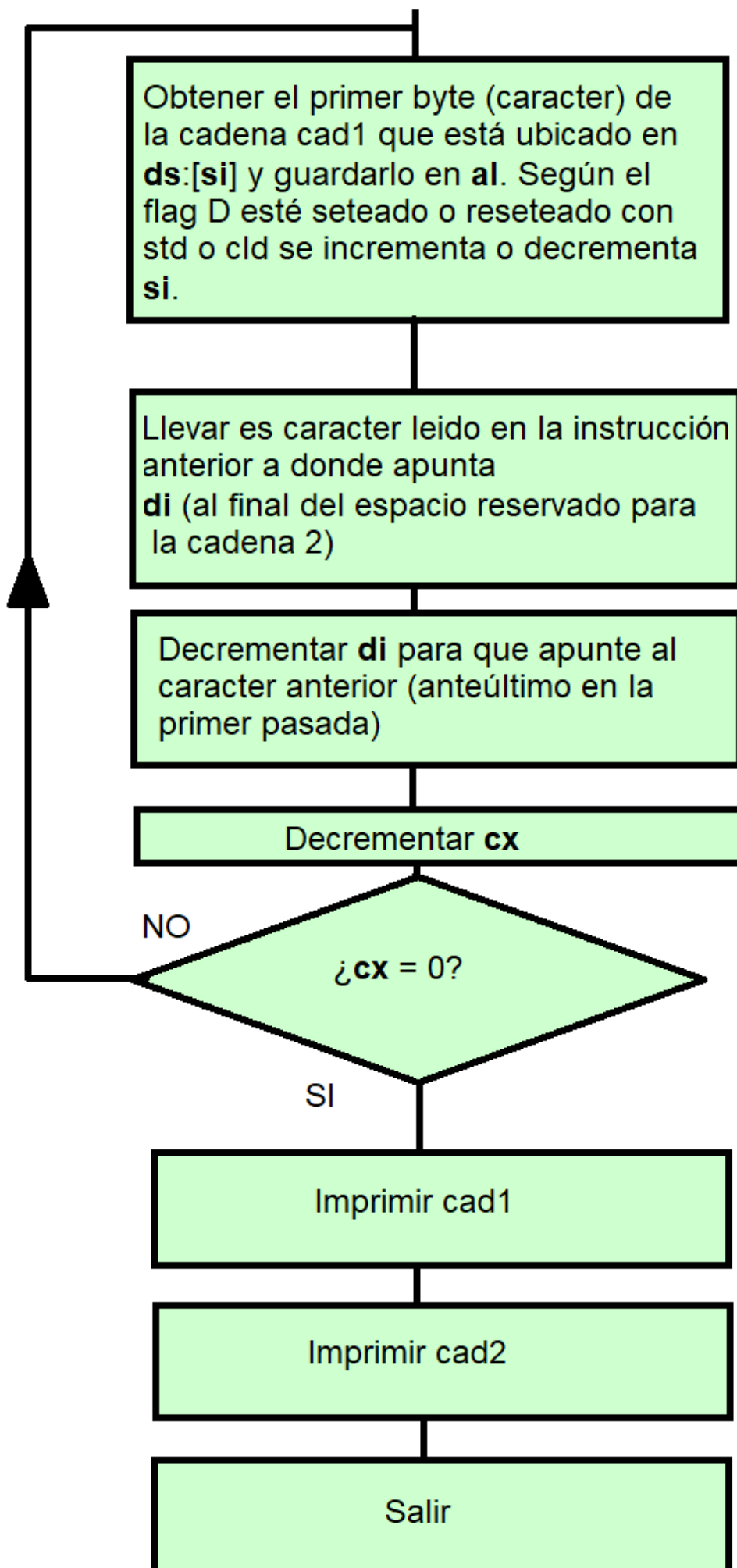
Cargar los data segment ds y es con la dirección del segmento en los que están los datos: variables cad1 y cad2 de las cadenas.

Fijar el flag de dirección D en 0 para que las instrucciones de manipulación de cadenas la recorran de izquierda a derecha.

Cargar en cx la longitud de la cadena para que la instrucción loop la vaya decrementando.

Las cadenas usan **si** y **di** para direccionar la memoria. **si** es un desplazamiento de **ds** y **di** es un desplazamiento para **es**.  
Cargar **di** con la dirección de inicio de la cadena cad1.

Cargar di con la dirección del último carácter de la cadena cad2 que es el primero de cad1.



# Programa

.model small

.stack 64

.DATA

cad1 db 'Cadena de prueba','\$'

cad2 db 16 dup (' '),'\$' ;El operador dup asigna 16 bytes para una cadena de caracteres

.CODE

inicio: ;Punto de entrada al programa

mov ax, @data ;inicia registros de segmento

mov ds, ax ;DS trabaja con SI DS:[SI]

mov es, ax ;ES trabaja con DI ES:[DI]

cld ;Procesamiento de izq->der.

;ESI y EDI se incrementan

;(STD los decrementa izq <-- der)

mov cx,16 ;Longitud de la cadena.

;Loop la va decrementando

;hasta 0 para saltar.

lea si,cad1 ;DS:SI->Cad1

;(load effective address)

;mueve a si el offset de cad1

lea di,cad2+15 ;ES:DI apuntan al final del

;área reservada para almacenar

;la cadena invertida

otro: ;

lodsb ;Obtener el primer caracter de Cad1.

;Carga un byte desde DS:[SI] en AL

;y actualiza SI.(+1 según flag DF)

mov [di],al ;almacenar el caracter leído de cad1

;en la posición actual de DI. Posición

; final de cad2 para el primer caracter.

dec di ;Decrementar di en 1

loop otro ;Obtener siguiente caracter de Cad1

```
lea dx,cad1 ;
mov ah,09h ;Imprimir cadena original
int 21h
```

```
lea dx,cad2 ; mueve a dx el offset de cad2
mov ah,09h ;Imprimir cadena invertida
int 21h ;
```

```
mov ax,4c00h ;Terminar programa y regresar al DOS
int 21h ;
END inicio
END
```

DIRECTIVAS:  
.model small  
.stack 64  
.DATA  
cadena 1 de origen y  
cadena 2 invertida (reservar espacio)

```
.model small
.stack 64
.DATA
cad1 db 'Cadena de prueba', '$'
cad2 db 16 dup (' '), '$'
```

Cargar los data segment ds y es con la dirección del segmento en los que están los datos: variables cad1 y cad2 de las cadenas.

```
mov ax, @data ;inicia registros de segmento
mov ds, ax ;DS trabaja con SI DS:[SI]
mov es, ax ;ES trabaja con DI ES:[DI]
```

Fijar el flag de dirección D en 0 para que las instrucciones de manipulación de cadenas la recorran de izquierda a derecha.

```
cld ;Procesamiento de izq->der.
;SI y DI se incrementan
;(STD los decremента izq <--- der)
```

Cargar en cx la longitud de la cadena para que la instrucción loop la vaya decrementando.

```
mov cx, 16 ;Longitud de la cadena.
;Loop la va decrementando
;hasta 0 para saltar.
```

Las cadenas usan **si** y **di** para direccionar la memoria. **si** es un desplazamiento de **ds** y **di** es un desplazamiento para **es**. Cargar **di** con la dirección de inicio de la cadena cad1.

```
lea si, cad1 ;DS:SI->Cad1
; (load effective address)
;mueve a si el offset de cad1
```

Cargar di con la dirección del último carácter de la cadena cad2 que es el primero de cad1.

```
lea di, cad2+15 ;ES:DI apuntan al final del
;area reservada para almacenar
;la cadena invertida
```

